ĐỀ TÀI 19

Thiết kế CSDL hệ thống đấu giá trực tuyến

Kiểm soát trả giá đồng thời
Khóa dòng và SERIALIZABLE có thử lại

BÁO CÁO ĐỒ ÁN CƠ SỞ DỮ LIỆU NÂNG CAO

PostgreSQL 18 - Python 3.12 - Dữ liệu giả lập

Phạm vi: chỉ bài 19 của tài liệu hướng dẫn được cung cấp.

Thông tin sinh viên, lớp và giảng viên: bổ sung theo thực tế trước khi nộp.

Ngày tạo: 27/10/2026 22:06 (Việt Nam)

Mục lục

Đề tài 19: Thiết kế CSDL hệ thống đấu giá trực tuyến và kiểm soát trả giá đồng thời . . . 4

Tóm tắt . . . 4

1. Bài toán, mục tiêu và phạm vi . . . 4

2. Đặc tả nghiệp vụ . . . 4

3. Thiết kế EER và lược đồ quan hệ . . . 6

4. Phụ thuộc hàm, khóa và chuẩn hóa . . . 6

5. SQL nâng cao và hàm nghiệp vụ . . . 9

6. Thiết kế transaction và kiểm soát đồng thời . . . 9

7. Mô hình đối tượng và OQL . . . 11

8. Thiết kế thực nghiệm . . . 11

9. Minh chứng tính đúng . . . 13

10. Hướng dẫn vận hành và hướng phát triển . . . 14

Tài liệu tham khảo . . . 14

Phụ lục A - Mô hình đối tượng và OQL **15**

1. Phân biệt phạm vi thực thi . . . 15

2. Mô hình lớp . . . 15

3. ODL rút gọn . . . 15

4. Truy vấn OQL . . . 15

5. Đối chiếu SQL . . . 16

Phụ lục B - Thao tác pgAdmin **17**

1. Mở đúng CSDL . . . 17

2. Khi cài trên CSDL mới . . . 17

3. Đọc dữ liệu mẫu . . . 17

4. Demo tự động trước khi demo thủ công . . . 17

5. Hai Query Tool để quan sát khóa dòng . . . 18

6. Gửi lại cùng UUID . . . 19

7. Thử SERIALIZABLE đúng cách . . . 19

8. Đóng phiên và kiểm chứng . . . 19

9. Các lỗi thường gặp . . . 19

Phụ lục C - Kịch bản trình bày **21**

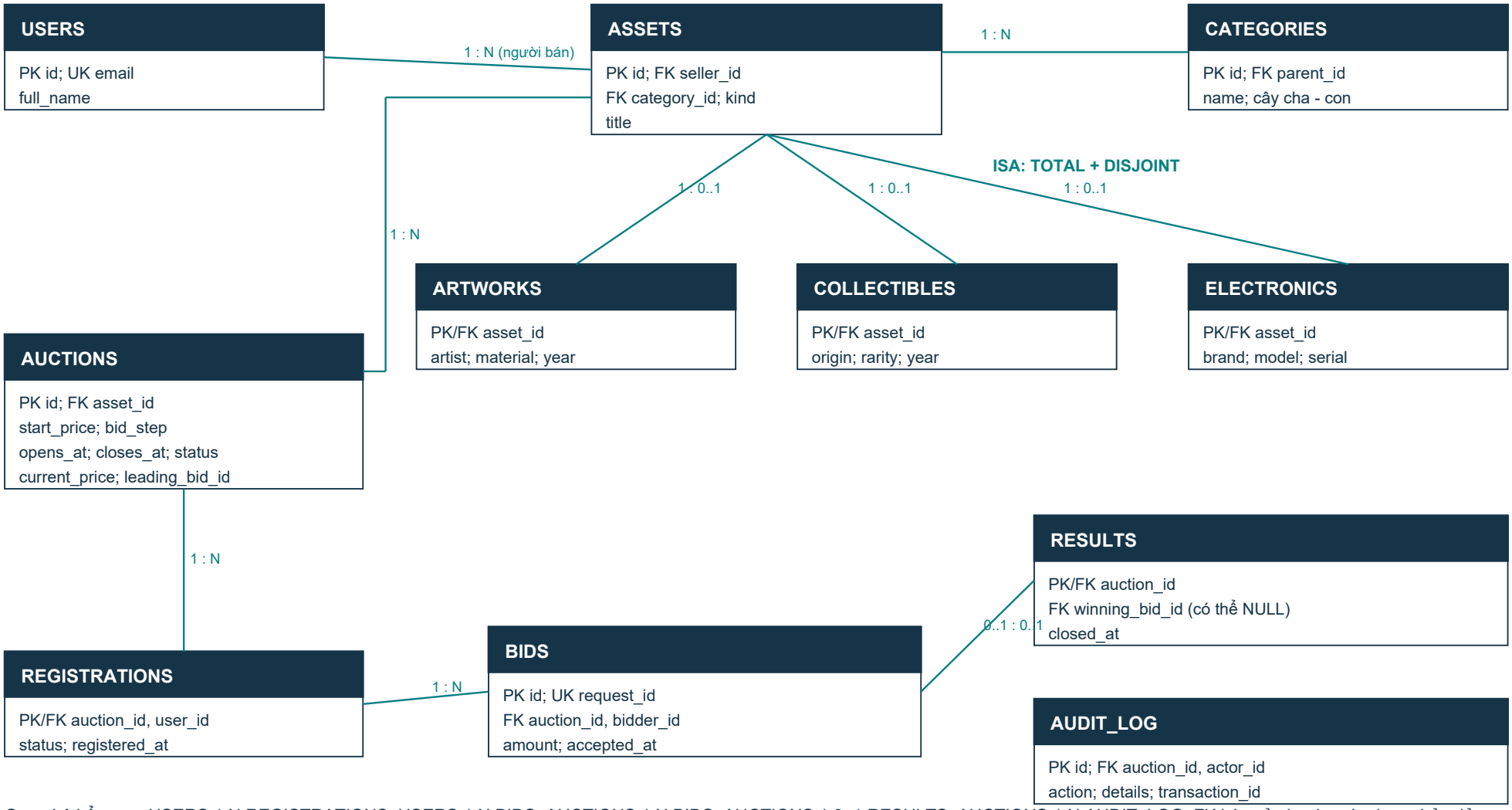
1. Trình bày khoảng 10-15 phút . . . 21

2. Câu hỏi thường gặp . . . 22

3. Trước khi nộp . . . 23

EER / LƯỢC ĐỒ QUAN HỆ - ĐỀ TÀI 19

PK: khóa chính | UK: duy nhất | FK: khóa ngoại | N: nhiều



Quan hệ bổ sung: USERS 1:N REGISTRATIONS; USERS 1:N BIDS; AUCTIONS 1:N BIDS; AUCTIONS 1:0..1 RESULTS; AUCTIONS 1:N AUDIT_LOG. FK kép của leader và winner bảo đảm lượt trả thuộc đúng phiên. Mỗi tài sản có đúng một lớp con; một người có thể bán ở phiên này và tham gia phiên khác.

BÁO CÁO ĐỒ ÁN CƠ SỞ DỮ LIỆU NÂNG CAO

Đề tài 19: Thiết kế CSDL hệ thống đấu giá trực tuyến và kiểm soát trả giá đồng thời

Nền tảng: PostgreSQL 18, Python 3.12, psycopg 3. Dữ liệu trong đồ án là dữ liệu giả lập. Thông tin sinh viên, lớp và giảng viên được người nộp bổ sung theo thông tin thực tế.

Tóm tắt

Đồ án giải quyết việc nhiều người trả giá cho cùng một tài sản, trong khi một giao dịch khác có thể đang đóng phiên. Nếu ứng dụng đọc giá hiện tại rồi ghi giá mới mà không điều phối giao dịch, một yêu cầu có thể kiểm tra trên dữ liệu cũ, làm mất cập nhật, hoặc tạo kết quả không tương ứng với lịch sử. Giải pháp sử dụng CSDL quan hệ chuẩn hóa, giữ lịch sử trả giá bất biến, thực hiện nghiệp vụ nguyên tử trong PostgreSQL và so sánh hai phương án trên cùng loại tải: khóa dòng phiên ở READ COMMITTED và SERIALIZABLE có thử lại toàn giao dịch.

Các tiêu chí kiểm chứng là: tối đa một kết quả cho mỗi phiên, giá hiện tại khớp lịch sử, cùng mã yêu cầu không sinh hai lượt trả giá, và lỗi giữa giao dịch không để lại dữ liệu dở dang. Kết quả benchmark được đo từ chương trình đi kèm, không lấy từ số liệu giả định.

1. Bài toán, mục tiêu và phạm vi

1.1. Câu hỏi nghiên cứu

Làm thế nào để mỗi lượt trả giá được kiểm tra trên trạng thái hợp lệ, không làm mất cập nhật và không tạo hai người thắng cho cùng một phiên khi nhiều yêu cầu chạy đồng thời?

1.2. Phạm vi

Quản lý người dùng, cây danh mục tài sản, tài sản có chuyên biệt hóa, phiên đấu giá, đăng ký tham gia, lịch sử trả giá, kết quả và nhật ký. Mô phỏng đấu giá tăng dần công khai, một tài sản trong một phiên, một đơn vị tiền tệ. Không triển khai thanh toán, giao hàng, giấy phép đấu giá, xác minh danh tính, chống thông đồng hoặc ứng dụng thương mại đầy đủ. Không triển khai bài 20 về RBAC.

1.3. Sản phẩm bàn giao

Mã SQL tạo và kiểm tra CSDL; dữ liệu mẫu; hàm trả giá/đóng phiên/báo cáo; truy vấn nâng cao; client Python; kiểm thử đồng thời; benchmark tái lập được; mô hình EER, đối tượng và OQL; hướng dẫn pgAdmin; báo cáo và kịch bản bảo vệ.

2. Đặc tả nghiệp vụ

2.1. Giá và bước giá

Giá khởi điểm `start_price` dương. Bước giá `bid_step` dương. Dùng `numeric(18,2)` để lưu tiền chính xác đến hai chữ số thập phân, phía Python dùng `Decimal`.

Nếu chưa có lượt trả giá hợp lệ, mức tối thiểu là `start_price`. Nếu đã có, mức tối thiểu là `current_price + bid_step`. Mọi mức giá phải thuộc lưới `start_price + k * bid_step`, với `k` nguyên không âm. Vì vậy không chỉ

kiểm tra lớn hơn giá trước mà còn phải kiểm tra đúng bước giá. Các trường hợp số âm, độ chính xác vượt quy định, NaN hoặc vô cực phải bị từ chối trước khi lưu.

`start_price` là điều kiện của phiên. `bids.amount` là giá của một lượt đã được chấp nhận. `current_price` là giá cao nhất suy diễn từ lịch sử; khi chưa có bid thì bằng NULL. Giá thắng là `amount` của `results.winning_bid_id`, không lưu thêm một bản sao giá thắng.

2.2. Mở, đóng và hạn nhận giá

Phiên có `opens_at < closes_at`. Lượt trả giá chỉ hợp lệ trong khoảng nửa mở `[opens_at, closes_at)` trên đồng hồ CSDL. Đúng `closes_at` đã hết hạn. Phiên có thể còn mang trạng thái `OPEN` sau hạn nếu tác vụ đóng chưa chạy; hàm trả giá vẫn từ chối bằng kiểm tra thời gian.

Thời điểm xét nhận là `clock_timestamp()` khi hàm được quyền xử lý phiên. Một yêu cầu gửi từ máy khách trước hạn nhưng chờ khóa đến sau hạn vẫn bị từ chối. Không tin thời gian do máy khách gửi. Không dùng `now()` để quyết định hạn sau khi chờ khóa, vì `now()` cố định từ đầu transaction và có thể đã cũ.

`close_auction` chỉ đóng khi thời gian máy chủ đạt `closes_at`; chưa tới hạn trả `TOO_EARLY`. Không có chức năng ép đóng trong hàm nghiệp vụ chính. Đóng lại một phiên đã đóng trả kết quả cũ.

2.3. Điều kiện người tham gia

Người dùng phải có đăng ký `APPROVED` của chính phiên. Chủ sở hữu tài sản không được tự trả giá. Vai trò người bán/người tham gia là quan hệ theo từng phiên, không phải hai lớp người dùng loại trừ nhau.

Trong thời gian thí nghiệm, cấu hình phiên và đăng ký được chuẩn bị trước. Tài khoản ứng dụng chỉ gọi hàm, không sửa trực tiếp tài sản, đăng ký, lịch sử hoặc kết quả. Tài khoản sở hữu CSDL dùng cho cài đặt và sinh dữ liệu không phải tài khoản ứng dụng thông thường.

2.4. Hai yêu cầu cùng giá và yêu cầu gửi lại

Hai yêu cầu khác UUID cùng mức giá không cùng được chấp nhận. Yêu cầu được xử lý hợp lệ trước cập nhật leader; yêu cầu sau phải kiểm tra lại giá mới và bị từ chối. Không bảo đảm người bấm nút trước hoặc gói mạng đến trước luôn thắng; thứ tự được xác định bởi quá trình xử lý giao dịch của CSDL.

UUID `request_id` nhận diện một yêu cầu trả giá trên toàn hệ thống. Nếu UUID đã được chấp nhận và nội dung `(auction_id, bidder_id, amount)` giống nhau, trả lại bid cũ, kể cả phiên đã đóng. Nếu UUID trùng nhưng nội dung khác, trả `IDEMPOTENCY_CONFLICT`. Cơ chế khóa request/định danh duy nhất phải xử lý cả trường hợp hai phiên khác nhau đồng thời sử dụng cùng UUID.

Chỉ yêu cầu được chấp nhận được lưu trong `bids`. Từ chối nghiệp vụ không được cache như một kết quả vĩnh viễn: chẳng hạn người dùng có thể được duyệt đăng ký sau đó. Vì vậy UUID của một yêu cầu đã bị từ chối có thể được thử lại và đánh giá trên trạng thái mới; client nên tạo UUID mới khi chủ động thay đổi ý định. Đây là idempotency của **thao tác đã ghi nhận**, không phải sổ lưu mọi lần gọi API.

2.5. Kết quả và kiểm toán

Không có giá dự trữ. Nếu có bid hợp lệ, người thắng là người của bid cao nhất. Nếu không có bid, vẫn tạo kết quả với `winning_bid_id IS NULL`, thể hiện phiên không thành công. Mỗi phiên kết thúc có đúng một dòng kết quả; trước khi đóng có thể chưa có dòng nào.

Ghi `BID_ACCEPTED` và `AUCTION_CLOSED` cùng giao dịch nghiệp vụ. Nhật ký là chứng cứ thay đổi đã commit; các yêu cầu bị từ chối và số lần retry được lưu ở log thực nghiệm phía client. Nếu giao dịch rollback, bid, cache

và audit đều rollback.

3. Thiết kế EER và lược đồ quan hệ

3.1. Thực thể và chuyên biệt hóa

Mô hình EER chi tiết trong `docs/EER.md`. Tài sản là lớp cha, có ba lớp con nghệ thuật, sưu tầm, điện tử; chuyên biệt hóa total và disjoint. Các bảng con dùng PK đồng thời là FK tới tài sản cha. Trigger ràng buộc hoãn đến cuối giao dịch kiểm tra mỗi tài sản có đúng bảng con phù hợp. Cần hoãn vì khi chèn tài sản cha, bảng con có thể chưa được chèn ở câu lệnh đó.

Cây danh mục dùng adjacency list `(id, parent_id)`, cấm chu trình. Người dùng bán tài sản qua `seller_id`; tham gia qua bảng nối `registrations`. Không tạo hai bản ghi người dùng cho hai vai trò.

3.2. Lược đồ vật lý

- `users(id PK, email UNIQUE, full_name, created_at).`
- `categories(id PK, parent_id FK -> categories, name);` tên trong cùng một cha duy nhất, kể cả các gốc cùng NULL.
- `assets(id PK, seller_id FK, category_id FK, kind, title, description, created_at).`
- `artworks(asset_id PK/FK, artist, material, creation_year).`
- `collectibles(asset_id PK/FK, origin, rarity, production_year).`
- `electronics(asset_id PK/FK, brand, model, serial_number UNIQUE nullable, condition).`
- `auctions(id PK, asset_id FK, start_price, bid_step, opens_at, closes_at, status, current_price, leading_bid_id FK, revision).`
- `registrations(auction_id PK/FK, user_id PK/FK, status).`
- `bids(id PK, auction_id FK, bidder_id FK, amount, request_id UNIQUE, accepted_at).`
- `results(auction_id PK/FK, winning_bid_id FK nullable, closed_at).`
- `audit_log(id PK, auction_id FK, action, actor_id FK nullable, details JSONB, occurred_at, transaction_id).`

Khóa ghép đăng ký ngăn một người đăng ký trùng phiên. UUID duy nhất ngăn một yêu cầu được lưu hai lần. FK kép `(leading_bid_id,id)` và `(winning_bid_id,auction_id)` tham chiếu `(bids.id,bids.auction_id)`, bảo đảm lượt dẫn/thắng thuộc đúng phiên; một FK chỉ vào `bids.id` không tự bảo đảm điều đó. Trigger kiểm chứng liên bảng bổ sung tính nhất quán leader, max-price và kết quả.

3.3. Chỉ mục

PK/UNIQUE tạo các chỉ mục tương ứng. Chỉ mục truy vấn phục vụ tìm phiên theo trạng thái/hạn đóng, tìm bid theo phiên và giá, tra đăng ký theo khóa ghép, duyệt con theo `categories.parent_id`, tìm tài sản theo danh mục. Đánh đổi là tăng dung lượng và chi phí ghi; chỉ mục không thay thế transaction hay kiểm tra nghiệp vụ.

4. Phụ thuộc hàm, khóa và chuẩn hóa

4.1. Quan hệ quan sát ban đầu

Xét quan hệ rộng `R` cho **mỗi lượt trả giá đã chấp nhận** gồm:

```
bid_id, request_id, auction_id, bidder_id, amount, accepted_at,
asset_id, start_price, bid_step, opens_at, closes_at,
seller_id, category_id, title, kind,
bidder_email, bidder_name, seller_email, seller_name,
category_name, parent_id
```

Tên và email được đặt hai bí danh để biểu diễn hai vai trò của cùng thực thể người dùng. Quan hệ R chỉ là công cụ phân tích; không thể biểu diễn phiên chưa có bid hoặc người chỉ đăng ký mà chưa trả giá, nên các thực thể đó vẫn được mô hình hóa riêng.

4.2. Tập phụ thuộc hàm

Từ quy tắc định danh, có tập F sau, viết gộp vế phải để dễ đọc:

```
bid_id -> request_id, auction_id, bidder_id, amount, accepted_at
request_id -> bid_id
auction_id -> asset_id, start_price, bid_step, opens_at, closes_at
asset_id -> seller_id, category_id, title, kind
bidder_id -> bidder_email, bidder_name
bidder_email -> bidder_id
seller_id -> seller_email, seller_name
seller_email -> seller_id
category_id -> category_name, parent_id
(auction_id, amount) -> bid_id
(parent_id, category_name) -> category_id
```

Không giả định họ tên là duy nhất, tên tài sản xác định tài sản, tên danh mục xác định danh mục toàn cục, hay mức giá xác định một bid trên toàn CSDL. Việc cấm hai bid bằng giá tạo khóa nghiệp vụ `(auction_id,amount)` trong dữ liệu bid hợp lệ, không phải FD `amount -> bid_id`. Khóa này được bảo đảm bởi protocol giá tăng nghiêm ngặt; schema không khai báo thêm UNIQUE cho cặp này. Với danh mục gốc, FD theo cặp cha/tên được hiểu trong mô hình logic dùng một ký hiệu gốc thống nhất; PostgreSQL hiện thực quy tắc tương đương bằng UNIQUE NULLS NOT DISTINCT để xử lý NULL của parent_id.

4.3. Khóa ứng viên bằng bao đóng

`{bid_id}+` chứa request, phiên, bidder, giá, thời gian; từ phiên suy ra tài sản/cấu hình; từ tài sản suy ra seller/danh mục; từ hai người và danh mục suy ra mọi thông tin mô tả. Vì vậy `bid_id` là khóa ứng viên.

`request_id -> bid_id`, nên `{request_id}+ = R`; `request_id` là khóa ứng viên thứ hai. Cả hai là thuộc tính đơn nên tối thiểu. Khóa thứ ba là `(auction_id,amount)` vì cặp này suy ra bid_id; từng thuộc tính riêng không suy ra bid_id nên cặp là tối thiểu. Các FD còn lại không sinh auction_id hoặc amount nếu thiếu các định thức này; vì vậy ba khóa trên là các khóa ứng viên theo F đã công bố.

4.4. Phủ tối tiểu

Tách từng phụ thuộc gộp vế phải ở mục 4.2 thành các FD có **một thuộc tính vế phải**. Tập thu được là phủ tối tiểu `Fmin`:

```
bid_id -> request_id       bid_id -> auction_id
bid_id -> bidder_id        bid_id -> amount
bid_id -> accepted_at      request_id -> bid_id
auction_id -> asset_id     auction_id -> start_price
auction_id -> bid_step     auction_id -> opens_at
auction_id -> closes_at    asset_id -> seller_id
asset_id -> category_id    asset_id -> title
asset_id -> kind           bidder_id -> bidder_email
bidder_id -> bidder_name   bidder_email -> bidder_id
seller_id -> seller_email  seller_id -> seller_name
seller_email -> seller_id  category_id -> category_name
category_id -> parent_id
```

```
(auction_id,amount) -> bid_id
(parent_id,category_name) -> category_id
```

Các FD có vế trái đơn không có thuộc tính thừa ở vế trái. Ở hai vế trái ghép, riêng auction_id hoặc amount không suy ra bid_id; riêng parent_id hoặc category_name không suy ra category_id. Khi bỏ từng FD, tính bao đóng của vế trái trên phần còn lại không sinh lại thuộc tính vế phải của FD đó. Ví dụ bỏ `auction_id -> start_price` thì không còn đường nào từ auction đến start_price; bỏ `request_id -> bid_id` thì request không sinh bid; bỏ `bidder_email -> bidder_id` thì email không sinh người dùng. Các cặp ID/email hai chiều đều cần để giữ khóa thay thế. Fmin tương đương F do phép tách/hợp vế phải.

4.5. Bất thường của quan hệ rộng

R không đạt 3NF vì `auction_id`, `asset_id`, `bidder_id`, `seller_id`, `category_id` không là siêu khóa của R nhưng xác định các thuộc tính không khóa. Thay email người bán phải sửa nhiều dòng; thêm phiên chưa có bid không có dòng đại diện; xóa bid cuối có thể mất thông tin phiên nếu chỉ lưu R. Các FD bắc cầu từ bid tới phiên, tài sản và mô tả tạo lặp thông tin.

4.6. Phân rã và chứng minh

Gom các FD chung vế trái để tạo `Bid`, `AuctionCore`, `Asset`, `Bidder`, `Seller`, `Category`. Hợp hai quan hệ `Bidder` và `Seller` về thực thể chuẩn `Users` với bí danh khi join. `Bid` chứa cả hai khóa bid_id/request_id. Mỗi định thức không tầm thường của các quan hệ trên là khóa ứng viên, nên đạt BCNF theo tập FD đã công bố.

Phân rã không mất mát: lần lượt tách nhóm thuộc tính do `auction_id`, `asset_id`, `bidder_id`, `seller_id`, `category_id` xác định. Mỗi bước hai quan hệ có giao là một định thức X; X là khóa của quan hệ được tách ra, thỏa điều kiện lossless của phân rã nhị phân `X -> Y`. Lặp các bước bảo toàn lossless; join hai bản sao Users theo đúng bí danh bidder/seller phục hồi R.

Bảo toàn phụ thuộc: từng FD của Fmin nằm trọn trong một quan hệ sau phân rã; kiểm tra tại quan hệ đó không cần join. Khóa thay thế email và request_id được giữ bằng UNIQUE. Các phụ thuộc được bảo toàn này là FD; những ràng buộc như thời hạn, giá tăng, cây không chu trình và winner cùng phiên vẫn cần CHECK/FK/trigger/hàm, không suy ra chỉ từ chuẩn hóa.

4.7. Các quan hệ bổ sung

`registrations`: `(auction_id,user_id) -> status,registered_at`, khóa ghép là định thức; đạt BCNF theo FD nghiệp vụ. `results`: `auction_id -> winning_bid_id,closed_at`, khóa là auction_id; không lặp tên và giá thắng. Các bảng con tài sản có FD `asset_id -> các thuộc tính chuyên biệt`, đạt BCNF; serial_number không NULL là khóa thay thế có điều kiện của thiết bị khi được khai báo duy nhất, không coi nhiều NULL là cùng một số serial. `audit_log`: `id -> mọi thuộc tính còn lại`; timestamp và transaction_id không phải khóa vì một giao dịch có thể có nhiều sự kiện. Tên danh mục chỉ duy nhất trong một cha; không coi `name` là khóa toàn cục.

4.8. Cache là phi chuẩn hóa có kiểm soát

Lược đồ logic chuẩn hóa không cần `current_price` hoặc `leading_bid_id`, vì có thể tính từ bids. Lược đồ vật lý giữ chúng để kiểm tra giá và hiển thị nhanh. FD nội bảng của auction vẫn do ID xác định, nhưng **dữ liệu suy diễn bị lặp xuyên bảng**; việc gọi một bảng là BCNF không tự chứng minh cache đúng.

Mọi thao tác thêm bid, cập nhật cache và audit cùng transaction. Không cho app sửa cache hoặc lịch sử trực tiếp. Trigger cuối giao dịch và hàm `verify_invariants()` đối chiếu cache với lịch sử. Sau lỗi, rollback trả cả lịch sử/cache về trạng thái trước. Không cho xóa/sửa bid đã chấp nhận; sửa nghiệp vụ phải có quy trình mới, ngoài phạm vi đồ án.

5. SQL nâng cao và hàm nghiệp vụ

5.1. Truy vấn phân tích

`sql/04_queries.sql` cung cấp các nhóm truy vấn cần thiết:

- Phiên đang nhận giá và sắp kết thúc: lọc status, khoảng thời gian và sắp theo closes_at.

- Người đang dẫn: join auctions -> bids qua leading_bid_id -> users.

- Lịch sử thay đổi giá: dùng `LAG(amount)` trong partition từng phiên để tính chênh lệch, thứ tự thời gian/ID ổn định.

- Số người tham gia: nhóm registrations; phân biệt tổng đăng ký, đã duyệt và số bidder đã thực sự trả giá.

- Tỷ lệ thành công: số phiên đã đóng có winning_bid_id chia cho tổng số phiên đã đóng; tránh chia cho 0 và không tính phiên còn mở vào mẫu số.

- Tổng hợp cây danh mục: `WITH RECURSIVE` tạo tập hậu duệ rồi tổng hợp. Khi join đồng thời đăng ký và bids phải tổng hợp riêng hoặc `COUNT(DISTINCT ...)`, tránh nhân dòng.

Ví dụ truy vấn leader:

```
SELECT a.id AS auction_id, u.full_name, b.amount
FROM auction.auctions AS a
LEFT JOIN auction.bids AS b ON b.id = a.leading_bid_id
LEFT JOIN auction.users AS u ON u.id = b.bidder_id
ORDER BY a.id;
```

LEFT JOIN giúp vẫn thấy phiên chưa có bid. Người đang dẫn chưa phải người thắng chính thức.

5.2. Báo cáo có bộ lọc động

`auction.auction_report(p_status, p_category_id, p_min_price)` nhận bộ lọc tùy chọn; NULL nghĩa là bỏ lọc. Danh mục có thể bao gồm hậu duệ bằng recursive CTE. Lọc được tham số hóa; không ghép chuỗi nhập của người dùng thành tên cột hay biểu thức SQL tùy ý. Đây là báo cáo động theo giá trị bộ lọc, không phải cho client thực thi SQL tự do.

```
SELECT * FROM auction.auction_report(NULL, NULL, NULL);
SELECT * FROM auction.auction_report('OPEN', NULL, 100000);
```

Giá được lọc là giá hiển thị `COALESCE(current_price,start_price)`: phiên chưa có bid được xét theo giá khởi điểm. Biểu thức nằm trực tiếp trong `sql/02_functions.sql`.

5.3. Giao diện hàm

```
SELECT auction.place_bid(1::bigint, 2::bigint, 100000::numeric,
    '00000000-0000-4000-8000-000000000001'::uuid, 'row_lock');
SELECT auction.close_auction(2::bigint, 'row_lock');
SELECT * FROM auction.verify_invariants();
```

Ví dụ số tiền chỉ minh họa cú pháp; mức hợp lệ phải lấy từ cấu hình của phiên. Mỗi transaction client chỉ gọi một thao tác nghiệp vụ. Kết quả JSON phân biệt chấp nhận, phát lại và từ chối nghiệp vụ. Lỗi hạ tầng/xung đột transaction là ngoại lệ cần rollback/retry, không phải lượt bị từ chối do giá thấp.

6. Thiết kế transaction và kiểm soát đồng thời

6.1. Rủi ro khi đọc rồi ghi rời rạc

Giả sử giá đang 100, bước 10. T1 và T2 cùng đọc 100. T1 nhận 120; T2 nhận 110 dựa trên bản đọc cũ. Nếu T2 ghi cache sau T1 thì cache về 110 trong khi lịch sử có 120. Nếu đóng phiên đọc leader cũ giữa các câu lệnh, kết quả có thể không khớp bid được chấp nhận. UNIQUE kết quả ngăn hai dòng nhưng không tự chọn đúng người thắng; cần điều phối toàn nghiệp vụ.

6.2. Phương án A: READ COMMITTED và khóa dòng

Trình tự rút gọn:

1. Mở transaction READ COMMITTED; đồng bộ request_id để nhận biết replay/xung đột UUID.
2. `SELECT ... FROM auctions WHERE id = ... FOR UPDATE.`
3. Sau khi lấy khóa, đọc trạng thái mới, giờ hiện tại, người bán và đăng ký; kiểm tra mức giá.
4. Chèn bid, cập nhật current_price/leading_bid_id/revision và ghi audit.
5. COMMIT, hoặc ROLLBACK tất cả nếu lỗi.

Khóa phiên giữ đến cuối giao dịch. Một transaction khác cùng phiên chờ; khi được chạy phải kiểm tra trạng thái/giá mới. Các phiên khác nhau không bị khóa chung bằng một khóa toàn hệ thống. Tất cả đường trả giá/đóng phiên phải dùng cùng protocol; ghi bảng trực tiếp sẽ phá giả thiết.

Giao dịch đóng cũng khóa cùng dòng phiên, kiểm tra đã đến hạn, chốt leader hoặc NULL, chèn results rồi chuyển CLOSED trong cùng transaction. Nếu bid có quyền xử lý trước hạn thì close sau đó thấy bid; nếu close giành quyền khi đã hết hạn thì bid chờ phải thấy CLOSED hoặc hết hạn. Không có cửa sổ giữa ghi kết quả và đổi trạng thái mà transaction khác được phép dựa vào dữ liệu dở dang.

6.3. Phương án B: SERIALIZABLE và thử lại

Client phải `BEGIN ISOLATION LEVEL SERIALIZABLE` **trước câu lệnh đọc đầu tiên**. Hàm đọc phiên bình thường, kiểm tra điều kiện và cập nhật phiên; không dùng `SELECT FOR UPDATE` trên phiên như phương án A. Khi lịch xử lý không thể tuần tự hóa hoặc hai transaction cập nhật trên snapshot không tương thích, PostgreSQL hủy một transaction.

Client bắt SQLSTATE `40001` (serialization_failure) và `40P01` (deadlock_detected), rollback, chờ backoff có jitter rồi tạo transaction mới, đọc lại toàn bộ và gọi lại hàm với **cùng request_id**. Không chạy lại riêng UPDATE trên snapshot cũ. Retry có giới hạn; nếu hết giới hạn ghi riêng `exhausted` để không che giấu lỗi thành công giả.

Đóng phiên cũng phải chạy trong transaction SERIALIZABLE. Hàm từ chối nếu client chọn strategy này nhưng isolation hiện tại không đúng. SERIALIZABLE không có nghĩa không sử dụng khóa: PostgreSQL vẫn có khóa ghi thông thường, snapshot và cơ chế SSI; khác biệt là không chủ động tuần tự hóa bằng khóa đọc dòng phiên trong mã nghiệp vụ.

6.4. Idempotency và điểm chấp nhận

Khóa/điều phối theo UUID là bổ sung cho khóa phiên: khóa phiên riêng lẻ không ngăn cùng UUID được đưa vào hai phiên khác nhau. UNIQUE request_id là lớp bảo vệ cuối. Cần so sánh payload sau khi tìm thấy UUID, không đơn giản trả thành công cho bất kỳ người gửi nào dùng lại UUID.

Thời điểm chấp nhận nghiệp vụ xảy ra tại bước kiểm tra thời gian hợp lệ trong transaction giành quyền xử lý; kết quả chỉ có hiệu lực bên ngoài sau COMMIT. Các transaction có thể chồng lấn về thời gian nên không diễn giải SERIALIZABLE là thứ tự tăng theo timestamp client. Tính đúng dựa vào bất biến sau mỗi commit và protocol đồng nhất.

6.5. So sánh lý thuyết

Tiêu chí	Khóa dòng	SERIALIZABLE + retry
Isolation client	READ COMMITTED	SERIALIZABLE
Cách bảo vệ phiên	Chủ động FOR UPDATE	Snapshot/SSI và xung đột cập nhật
Khi cạnh tranh	Chủ yếu chờ khóa	Có thể bị abort rồi làm lại
Mã client	Có xử lý lỗi transaction	Bắt buộc retry toàn transaction
Hotspot một phiên	Hàng đợi trên một dòng	Nhiều retry khi nhiều người cùng ghi
Quy tắc nhiều dòng	Phải xác định đầy đủ thứ tự khóa	Có thể phát hiện phụ thuộc đọc/ghi rộng hơn
Nguy cơ vận hành	Giữ khóa lâu, deadlock nếu sai thứ tự	Retry storm, exhausted khi đặt giới hạn thấp

Không kết luận tuyệt đối về tốc độ từ bảng lý thuyết. Cả hai cùng ghi một dòng trạng thái nên đều có hotspot. Cần đo latency gồm retry và tỷ lệ yêu cầu hoàn thành dưới cùng cấu hình.

7. Mô hình đối tượng và OQL

Các lớp bắt buộc `TaiSan`, `PhienDauGia`, `LuotTraGia`, `NguoiThamGia`, `KetQuaDauGia`, quan hệ và ví dụ ODL/OQL được trình bày trong `docs/MO_HINH_DOI_TUONG_OQL.md`. Truy vấn mẫu gồm tìm phiên theo loại tài sản, duyệt collection lượt trả giá, tìm leader qua giá lớn nhất và đọc người thắng qua kết quả.

PostgreSQL thực thi SQL; OQL chỉ là phần đặc tả mô hình đối tượng. Lệnh `oql` là bản mô phỏng truy vấn tương đương bằng Python trên các đối tượng mẫu, có thể chạy để đối chiếu kết quả. Không khẳng định chạy OQL nguyên bản trong pgAdmin. Định danh đối tượng được ánh xạ về PK; object relationship được ánh xạ về FK/join.

8. Thiết kế thực nghiệm

8.1. Môi trường và cách tái lập

Máy Windows, PostgreSQL 18 đang chạy cục bộ, Python 3.12 và psycopg 3. Phiên bản chi tiết và thông số workload được lưu trong kết quả thực nghiệm. Ba CSDL riêng: demo để trình bày, `_test` cho kiểm thử, `_bench` cho benchmark. Chương trình từ chối thao tác reset của test/benchmark nếu tên đích không phù hợp hoặc trùng CSDL demo.

```
.\.venv\Scripts\python.exe -m auction_lab test --output output/test-results.json
.\.venv\Scripts\python.exe -m auction_lab benchmark --workers 12 --requests 60 --rounds 3
--output output/benchmark
```

Mỗi strategy chạy cùng số người, cùng cấu hình giá, cùng dãy payload và số worker; dữ liệu được khởi tạo lại độc lập trước lượt đo. Thứ tự thực thi thực tế do bộ lập lịch quyết định, vì vậy số bid được chấp nhận có thể khác nhau dù dãy yêu cầu giống nhau. Barrier khởi động giúp tạo cạnh tranh thực, không dùng vòng lặp tuần tự giả đồng thời.

8.2. Các kịch bản kiểm thử

1. Giá hợp lệ/không hợp lệ, người chưa duyệt, người bán tự trả, phiên chưa mở và phiên hết hạn.
2. Nhiều bidder cùng trả một mức: đúng một bid được chấp nhận, các yêu cầu còn lại bị từ chối do giá.

3. Dãy mức giá cạnh tranh: max-cache luôn khớp MAX lịch sử, leader thuộc đúng phiên.

4. Gửi lại cùng UUID và cùng nội dung: một bid duy nhất, các lần sau replay; cùng UUID khác payload: conflict.

5. Hai phiên khác nhau đồng thời dùng cùng UUID: không tạo hai bid, không lộ replay sai payload.

6. Trả giá cạnh tranh với đóng phiên: kết quả thống nhất với lịch sử, tối đa một kết quả và không nhận bid sau hạn.

7. Chờ khóa đi qua closes_at: yêu cầu phải bị từ chối dù bắt đầu transaction trước hạn.

8. Nhiều giao dịch đóng cùng phiên: chỉ một kết quả; các lần còn lại trả lại kết quả đã chốt.

9. Đóng phiên không có bid: một kết quả với winning_bid_id NULL.

10. Lỗi sau INSERT bids, trước UPDATE cache: trigger chỉ cài ở CSDL test phát sinh lỗi; rollback không để lại bid/cache/audit dở dang. Không thêm cửa hậu fault injection vào hàm production.

11. Chọn strategy serializable ở isolation sai: bị từ chối; retry thực sự đi qua transaction mới.

12. Kiểm tra ràng buộc lớp con tài sản, vòng danh mục và tính bất biến lịch sử theo các test đi kèm.

8.3. Chỉ số đo

`accepted`: yêu cầu tạo bid mới; `replayed`: trả lại bid đã có; `rejected`: vi phạm nghiệp vụ; `retries`: tổng lần phải chạy lại transaction; `exhausted`: hết giới hạn thử lại; `errors`: lỗi ngoài các loại dự kiến. Số yêu cầu logic là số lần client được gọi; tổng attempts có thể lớn hơn vì retry.

Latency end-to-end đo từ lúc worker bắt đầu yêu cầu logic tới kết quả cuối, bao gồm tạo kết nối, chờ khóa, backoff và retry; không bao gồm thời gian request nằm chờ trong hàng đợi ThreadPool trước khi worker bắt đầu. Mean/p50/p95 tính trên tất cả yêu cầu, kể cả bị từ chối; percentile dùng nearest-rank `ceil(q*n)`. Không chia throughput cho tổng attempts khi đang gọi đó là throughput yêu cầu logic.

8.4. Kết quả thực đo

Thời điểm ghi nhận benchmark: 21:59:15 ngày 27/10/2026 theo giờ Việt Nam. Môi trường thực đo: PostgreSQL 18.6 x86_64 Windows, Python 3.12.10, psycopg 3.3.6, Windows 11; max_connections=100, shared_buffers=128MB, synchronous_commit=on. Nguồn: `output/benchmark/results.json`, `summary.csv`, `requests.csv`.

Workload có seed shuffle 19019, 12 worker, 60 yêu cầu mỗi strategy mỗi round, 3 round, tổng 360 yêu cầu. Thứ tự chạy strategy được đảo giữa các round để giảm thiên lệch làm nóng. Mỗi round dùng dữ liệu mới tương đương.

Round	Strategy	Accepted	Replayed	Rejected	Retry	Mean ms	p50 ms	p95 ms	Req/s
1	row_lock	4	2	54	0	100.547	91.267	197.510	103.919
1	serializable	7	3	50	17	126.342	90.339	298.295	62.881
2	row_lock	4	2	54	0	112.884	107.951	154.823	100.221
2	serializable	4	2	54	5	100.529	86.987	190.670	107.174
3	row_lock	3	2	55	0	106.092	100.553	159.757	96.058
3	serializable	7	3	50	17	105.331	76.813	256.966	91.223

Cả 6 đợt đều có errors=0, exhausted=0 và 8/8 bất biến đạt. Tổng theo strategy: row_lock có 11 accepted, 6 replayed, 163 rejected và 0 retry; serializable có 18 accepted, 8 replayed, 154 rejected và 39 retry. Mean

gộp 180 yêu cầu/strategy lần lượt khoảng 106.508 ms và 110.734 ms. Không lấy trung bình các p95 theo round để gọi đó là p95 gộp.

Bộ test cuối cùng: **35/35 đạt**, 0 failure, 0 error, 0 skip, thời gian 17.034 giây theo `output/test-results.json`, ghi nhận lúc 22:03:26 ngày 27/10/2026 theo giờ Việt Nam. Kiểm thử bao gồm lỗi tuần tự hóa thật và retry toàn transaction, không chỉ mô phỏng exception phía client. Kịch bản biên dùng Event để giữ một bid hợp lệ chưa commit qua hạn đóng, chứng minh close phải chờ và chốt đúng bid đó ở cả hai protocol; với SERIALIZABLE, close thực sự có retry.

Ở workload cục bộ này, row_lock có p95 thấp hơn ở cả ba round và không cần retry. SERIALIZABLE có 39 retry nhưng vẫn hoàn thành mọi yêu cầu logic; mean của round 2 và 3 có thể thấp hơn row_lock. Số accepted khác nhau vì giá cao có thể được xử lý sớm và làm những giá thấp còn lại bị từ chối. Các quan sát này không chứng minh ưu thế phổ quát ngoài cấu hình đã đo.

8.5. Phân tích và giới hạn suy luận

Khóa dòng thường phù hợp với nghiệp vụ có điểm tranh chấp rõ là một phiên: dễ kiểm tra thứ tự, nhưng giao dịch giữ khóa lâu làm tăng thời gian chờ. SERIALIZABLE giúp bảo vệ lịch đọc/ghi theo khả năng tuần tự hóa, đổi lại có thể cần nhiều retry dưới hotspot. Cả hai chỉ đúng khi mọi thao tác liên quan tuân thủ protocol và quyền truy cập không cho đi vòng qua hàm.

So sánh tốc độ phải đi kèm số accepted/rejected/retries/exhausted: một phương án trả lỗi nhanh không thể được coi là tốt hơn chỉ vì mean latency thấp. Một lần benchmark trên máy cá nhân không đủ kết luận cho tải internet, dữ liệu lớn hoặc nhiều máy chủ. Số người thắng là hệ quả của quy tắc và lịch cạnh tranh, không là bằng chứng duy nhất về tính đúng.

9. Minh chứng tính đúng

9.1. Tối đa một kết quả thắng

PK `results.auction_id` ngăn hai dòng kết quả. Giao dịch đóng phối hợp với trả giá trên cùng trạng thái phiên. Kiểm tra winner thuộc cùng phiên và bằng leader/max lịch sử; phiên không bid có winner NULL. Kiểm thử close đồng thời và close/bid kiểm chứng các trường hợp cạnh tranh.

9.2. Cache khớp lịch sử

Bid, cache và audit cập nhật nguyên tử. Lịch sử bất biến; app không được DML trực tiếp. Kiểm tra tổng hợp `MAX(amount)` và leading_bid_id sau các đợt cạnh tranh, cộng trigger ràng buộc cuối transaction. `verify_invariants()` cần mọi dòng passed=true; không chỉ nhìn dòng giá cuối trên giao diện.

9.3. Không nhân đôi yêu cầu

UNIQUE request_id và protocol theo UUID ngăn trùng khi gửi đồng thời. So sánh payload bảo đảm chỉ replay yêu cầu giống nội dung. Kiểm tra `COUNT(*)` theo request_id và giữ bid_id cũ khi replay sau đóng phiên.

9.4. Không có dữ liệu dở dang

Lỗi chèn có thể làm sequence ID bị khuyết vì sequence không rollback; khoảng trống ID không phải mất dữ liệu. Điều cần chứng minh là không có bid/audit mới đã commit và cache vẫn ở giá trước sau lỗi giữa transaction. Test fault injection kiểm tra dữ liệu trước/sau và thực hiện lại thành công khi bỏ trigger test.

10. Hướng dẫn vận hành và hướng phát triển

README cung cấp lệnh chạy; `docs/HUONG_DAN_PGADMIN.md` chỉ cách dùng Query Tool và quan sát chờ khóa bằng hai cửa sổ. `docs/BAO_VE.md` đề xuất thứ tự demo, giải thích từng bất biến và các câu hỏi thường gặp.

Tài khoản app chỉ có quyền đọc dữ liệu cần thiết và EXECUTE hàm nghiệp vụ; không cấp sửa bảng trực tiếp. SECURITY DEFINER phải cố định search_path an toàn, chỉ tham chiếu đối tượng tin cậy và thu hồi quyền EXECUTE mặc định PUBLIC khi cần. Chủ sở hữu CSDL vẫn là tài khoản quản trị tin cậy có thể thay đổi lược đồ; không coi ràng buộc trong ứng dụng là ranh giới chống superuser.

Hướng phát triển: quản lý duyệt đăng ký qua hàm có protocol rõ; quản lý vòng đời tài sản/phiên; job đóng phiên đến hạn; giao diện người dùng có xác thực; phân trang lịch sử; hàng đợi/thông báo sau commit; giám sát lock/retry; thêm thử nghiệm quy mô lớn và nhiều phiên. Thanh toán, đấu giá có giá dự trữ hoặc gia hạn chống sniping cần đặc tả lại quy tắc trước khi mở rộng.

Tài liệu tham khảo

1. Silberschatz, Korth, Sudarshan. *Database System Concepts*, 7th edition, slides chương 6-7 (E-R, chuẩn hóa), 17-19 (transaction, đồng thời, phục hồi), 29 (CSDL đối tượng): https://www.db-book.com/slides-dir/index.html .

2. PostgreSQL Global Development Group. PostgreSQL 18, *Explicit Locking*: https://www.postgresql.org/docs/18/explicit-locking.html .

3. PostgreSQL Global Development Group. PostgreSQL 18, *Transaction Isolation*: https://www.postgresql.org/docs/18/transaction-iso.html .

4. PostgreSQL Global Development Group. PostgreSQL 18, *WITH Queries*: https://www.postgresql.org/docs/18/queries-with.html .

5. PostgreSQL Global Development Group. PostgreSQL 18, *Constraints*: https://www.postgresql.org/docs/18/ddl-constraints.html .

Tài liệu giao đề là nguồn xác định phạm vi và các minh chứng cần nộp. Các quy tắc cụ thể như lưới giá, thời điểm xét sau chờ khóa, idempotency chỉ lưu lượt đã nhận và không có giá dự trữ là quyết định thiết kế được công bố trong báo cáo, không trình bày như yêu cầu nguyên văn của giảng viên.

Phụ lục A - Mô hình đối tượng và OQL

1. Phân biệt phạm vi thực thi

PostgreSQL của đồ án lưu dữ liệu quan hệ và chạy SQL/PLpgSQL. PostgreSQL **không có bộ thực thi ODMG OQL nguyên bản**. ODL/OQL dưới đây là đặc tả học thuật theo mô hình đối tượng; lệnh `python -m auction_lab oql` chạy các phép truy vấn Python tương đương trên đối tượng mẫu, không giả làm một OQL engine.

2. Mô hình lớp

Sơ đồ được trình bày ở trang EER; mã Mermaid đầy đủ có trong docs/EER.md.

`NguoiThamGia` là lớp người dùng có khả năng tham gia nghiệp vụ, không có nghĩa người này đã đăng ký mọi phiên. Quan hệ đăng ký được kiểm tra trong tầng nghiệp vụ/CSDL. Dùng `Decimal` cho tiền để tránh sai số nhị phân của `float`.

3. ODL rút gọn

Đây là ODL ở mức minh họa mô hình; khai báo kiểu tiền, thời gian và UUID có thể cần ánh xạ khác nhau tùy OODBMS.

```
interface NguoiThamGia (extent NguoiDung key id) {
  attribute long id;
  attribute string hoTen;
  attribute string email;
};
interface TaiSan (extent TatCaTaiSan key id) {
  attribute long id;
  attribute string ten;
  relationship NguoiThamGia nguoiBan;
};
interface TacPhamNgheThuat : TaiSan (extent TacPham) { attribute string tacGia; };
interface DoSuuTam : TaiSan { attribute string boSuuTap; };
interface ThietBiDienTu : TaiSan { attribute string hangSanXuat; };
interface PhienDauGia (extent Phien key id) {
  attribute long id;
  attribute double giaKhoiDiem; // Ky hieu minh hoa; trien khai dung Decimal.
  attribute double buocGia;
  attribute string trangThai;
  relationship TaiSan taiSan;
  relationship list<LuotTraGia> cacLuotTraGia
    inverse LuotTraGia::phien;
  relationship KetQuaDauGia ketQua inverse KetQuaDauGia::phien;
};
interface LuotTraGia (extent Luot key id) {
  attribute long id;
  attribute double soTien;
  attribute string maYeuCau;
  relationship PhienDauGia phien inverse PhienDauGia::cacLuotTraGia;
  relationship NguoiThamGia nguoiTra;
};
interface KetQuaDauGia (extent KetQua) {
  relationship PhienDauGia phien inverse PhienDauGia::ketQua;
  relationship LuotTraGia luotThang;
};
```

`double` trong ODL chỉ là kiểu chuẩn dễ đọc của ví dụ ngôn ngữ; không dùng nó cho lưu trữ tiền của chương trình. Một OODBMS hỗ trợ kiểu tiền có thể thay bằng decimal domain.

4. Truy vấn OQL

4.1. Phiên đấu giá tác phẩm nghệ thuật

Phép ép kiểu và kiểm tra kiểu lớp có thể khác nhau giữa các triển khai ODMG. Biến thể dưới đây dùng extent riêng của lớp con để tránh phụ thuộc vào toán tử `instanceof` không đồng nhất:

```
SELECT p
FROM p IN Phien, a IN TacPham
WHERE p.taiSan = a AND p.trangThai = "OPEN";
```

Trong đó `TacPham` là extent chứa các đối tượng `TacPhamNgheThuat`, còn `Phien` chứa `PhienDauGia`.

4.2. Các lượt trả giá của phiên 1

```
SELECT STRUCT(id: b.id, nguoiTra: b.nguoiTra.hoTen, gia: b.soTien)
FROM p IN Phien, b IN p.cacLuotTraGia
WHERE p.id = 1
ORDER BY b.soTien DESC;
```

4.3. Người đang dẫn đầu của phiên 1

```
SELECT STRUCT(nguoiDan: b.nguoiTra.hoTen, giaDan: b.soTien)
FROM p IN Phien, b IN p.cacLuotTraGia
WHERE p.id = 1
  AND b.soTien = MAX(SELECT x.soTien FROM x IN p.cacLuotTraGia);
```

Phiên không có lượt trả giá cho kết quả rỗng. Quy tắc giá tăng nghiêm ngặt khiến không có hai lượt được chấp nhận cùng mức giá trong một phiên.

4.4. Người thắng đã chốt

```
SELECT STRUCT(phien: k.phien.id,
              nguoiThang: k.luotThang.nguoiTra.hoTen,
              giaThang: k.luotThang.soTien)
FROM k IN KetQua
WHERE k.luotThang != nil;
```

Người dẫn đầu là trạng thái tạm thời; người thắng chỉ xác lập sau khi đóng phiên thành công. Không đồng nhất hai khái niệm.

5. Đối chiếu SQL

```
SELECT a.id, s.title
FROM auction.auctions a
JOIN auction.assets s ON s.id = a.asset_id
WHERE s.kind = 'ARTWORK' AND a.status = 'OPEN';
SELECT b.id, u.full_name, b.amount, b.accepted_at
FROM auction.bids b
JOIN auction.users u ON u.id = b.bidder_id
WHERE b.auction_id = 1
ORDER BY b.amount DESC;
SELECT u.full_name, b.amount
FROM auction.auctions a
JOIN auction.bids b ON b.id = a.leading_bid_id
JOIN auction.users u ON u.id = b.bidder_id
WHERE a.id = 1;
```

SQL dùng khóa và phép join; mô hình đối tượng dùng định danh đối tượng và đi theo relationship. Chương trình mô phỏng giúp kiểm tra kết quả truy vấn mẫu, nhưng việc chống mất cập nhật vẫn do PostgreSQL transaction đảm nhiệm.

Phụ lục B - Thao tác pgAdmin

1. Mở đúng CSDL

1. Mở pgAdmin 4, mở nhóm **Servers**, chọn PostgreSQL đang chạy trên máy.
2. Nếu pgAdmin hỏi mật khẩu PostgreSQL, nhập mật khẩu tài khoản bạn đã thiết lập khi cài; đây không phải mật khẩu GitHub hay mật khẩu Windows.
3. Mở **Databases**, chọn CSDL demo của đồ án (trên máy đã cài là `auction19_demo`), bấm phải và chọn **Query Tool**.
4. Chạy câu kiểm tra:

```
SELECT current_database(), current_user, version();
SELECT * FROM auction.auction_report(NULL,NULL,NULL);
SELECT * FROM auction.verify_invariants();
```

Chạy bằng nút Execute/F5. Nếu chọn một đoạn SQL, chỉ đoạn được chọn sẽ chạy. Trong kết quả verify, mọi dòng phải có `passed = true`.

2. Khi cài trên CSDL mới

Dùng CSDL riêng, tài khoản sở hữu lược đồ, chạy lần lượt `01_schema.sql`, `02_functions.sql`, `03_seed.sql` qua **Open File** trong Query Tool. `04_queries.sql` và `05_verify.sql` là truy vấn đọc/kiểm tra. Nếu dùng tài khoản ứng dụng, cấu hình riêng quyền theo script permissions và thông tin cài đặt; không cấp quyền ghi trực tiếp mọi bảng chỉ để demo.

Không chạy script tạo lại dữ liệu test/benchmark vào CSDL đang chứa dữ liệu thật. CLI `setup` từ chối nếu schema đã tồn tại. Nếu đã cài rồi chỉ cần chạy `list`, `demo`, `verify`.

Nếu cần chỉnh cây danh mục bằng tài khoản quản trị, dùng READ COMMITTED hoặc SERIALIZABLE có retry; trigger từ chối REPEATABLE READ để tránh đọc cây trên snapshot cũ sau khi chờ khóa. Thao tác này không thuộc đường trả giá của app.

3. Đọc dữ liệu mẫu

```
-- Cấu hình và giá đang dẫn
SELECT id,asset_id,start_price,bid_step,opens_at,closes_at,
       status,current_price,leading_bid_id
FROM auction.auctions ORDER BY id;
-- Lịch sử lượt trả giá
SELECT b.id,b.auction_id,u.full_name,b.amount,b.request_id,b.accepted_at
FROM auction.bids b JOIN auction.users u ON u.id=b.bidder_id
ORDER BY b.auction_id,b.amount;
-- Kết quả chính thức: NULL nghĩa là không có lượt thắng
SELECT r.auction_id,u.full_name AS winner,b.amount AS winning_price,r.closed_at
FROM auction.results r
LEFT JOIN auction.bids b ON b.id=r.winning_bid_id
LEFT JOIN auction.users u ON u.id=b.bidder_id
ORDER BY r.auction_id;
```

Giờ hiển thị có thể theo múi giờ session. Có thể đặt `SET TIME ZONE 'Asia/Ho_Chi_Minh';` để xem theo giờ Việt Nam; dữ liệu `timestamptz` vẫn biểu diễn cùng thời điểm tuyệt đối.

4. Demo tự động trước khi demo thủ công

Trong PowerShell tại thư mục dự án:

```
.\.venv\Scripts\python.exe -m auction_lab demo --strategy row_lock
```

```
.\.venv\Scripts\python.exe -m auction_lab demo --strategy serializable
```

Mỗi lần chạy tạo một phiên mới, giá khởi điểm 100, bước 10, hạn khoảng 4 giây. Chương trình thực hiện lượt đầu, replay cùng UUID, hai người cùng trả 110, đóng sớm, đóng đúng hạn, đóng lại và kiểm tra bất biến. Ghi lại `auction_id` trong JSON rồi lọc dữ liệu phiên đó trên pgAdmin. Demo tạo thêm dữ liệu, không reset schema demo.

5. Hai Query Tool để quan sát khóa dòng

5.1. Chuẩn bị phiên dành riêng cho thí nghiệm

Thực hiện bằng tài khoản sở hữu CSDL. Dữ liệu mẫu đã có asset 1 và users 2, 3. Tạo phiên mới và hai đăng ký trong một câu lệnh:

```
WITH new_auction AS (
  INSERT INTO auction.auctions
    (asset_id,start_price,bid_step,opens_at,closes_at)
  VALUES (1,100000,10000,
          clock_timestamp()-interval '1 minute',
          clock_timestamp()+interval '5 minutes')
  RETURNING id
), registrations AS (
  INSERT INTO auction.registrations(auction_id,user_id)
  SELECT a.id,u.id FROM new_auction a
  CROSS JOIN (VALUES (2::bigint),(3::bigint)) AS u(id)
  RETURNING auction_id
)
SELECT DISTINCT auction_id FROM registrations;
```

Ghi lại số trả về. Trong các câu tiếp theo thay `123` bằng ID thực tế. Không dùng ID 123 khi chưa tạo nó. Mở thêm Query Tool thứ hai cho **cùng CSDL**; hai tab phải là hai kết nối, không chỉ hai đoạn trong cùng một cửa sổ editor.

5.2. Tab A giữ khóa

Chạy riêng đoạn này, chưa COMMIT:

```
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT id,current_price FROM auction.auctions WHERE id=123 FOR UPDATE;
```

Tab A đang giữ khóa trên dòng phiên. Tránh để transaction quên mở lâu sau thí nghiệm.

5.3. Tab B yêu cầu cùng phiên

Chạy ở tab B:

```
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT auction.place_bid(123::bigint,3::bigint,110000::numeric,
                         gen_random_uuid(),'row_lock');
COMMIT;
```

Tab B sẽ chờ vì tab A chưa giải phóng khóa. Đây là chờ khóa thật, không phải ứng dụng bị treo. Nếu phiên đã hết 5 phút, tạo phiên thí nghiệm mới trước khi làm lại.

5.4. Tab A nhận giá rồi commit

Quay tab A, chạy:

```
SELECT auction.place_bid(123::bigint,2::bigint,110000::numeric,
                         gen_random_uuid(),'row_lock');
COMMIT;
```

Tab A nhận giá 110000. Sau COMMIT, tab B chạy tiếp, đọc trạng thái mới và trả `BID_TOO_LOW` vì không còn đủ mức tối thiểu 120000. Chỉ một bid giá 110000 tồn tại. Nếu tab B gửi 120000 thì có thể được nhận nếu vẫn trước hạn.

```
SELECT * FROM auction.bids WHERE auction_id=123;
SELECT * FROM auction.verify_invariants();
```

Nếu bấm Cancel khi đang chờ hoặc gặp lỗi SQL, chạy `ROLLBACK;` trên kết nối tương ứng rồi mới bắt đầu thí nghiệm khác. Cảnh báo “transaction is aborted” nghĩa là cần rollback giao dịch đang lỗi.

6. Gửi lại cùng UUID

Trong một phiên còn hạn và có đăng ký được duyệt, tạo một UUID rồi **giữ nguyên nó** khi gửi lại:

```
SELECT gen_random_uuid();
```

Sao chép UUID, lấy mức giá hợp lệ của phiên và chạy hai lần:

```
SELECT auction.place_bid(123::bigint,2::bigint,120000::numeric,
 '00000000-0000-4000-8000-000000000099'::uuid,'row_lock');
```

UUID trong ví dụ là minh họa, thay bằng UUID vừa tạo để tránh đụng dữ liệu của lần demo trước. Lần đầu `ACCEPTED`, lần sau `REPLAYED`, bid_id không đổi. Với cùng UUID nhưng số tiền/người/phiên khác, kết quả `IDEMPOTENCY_CONFLICT`. Nếu mức giá không hợp lệ ngay lần đầu thì không có bid được lưu và đó chưa phải trường hợp replay.

7. Thử SERIALIZABLE đúng cách

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT auction.place_bid(123::bigint,3::bigint,130000::numeric,
                         gen_random_uuid(),'serializable');
COMMIT;
```

Mức giá trong ví dụ cần điều chỉnh theo dữ liệu thực. Hai tab cùng cập nhật có thể làm một tab gặp `40001`; khi đó `ROLLBACK`, tạo transaction mới, đọc lại toàn trạng thái và gọi lại với cùng UUID yêu cầu. pgAdmin không tự thực hiện vòng retry. Dùng CLI/test/benchmark để xem cơ chế retry tự động và đo số lần retry.

Không thể đổi isolation sang SERIALIZABLE sau khi transaction đã đọc dữ liệu. Gọi strategy serializable trong READ COMMITTED bị hàm từ chối; lựa chọn strategy không tự nâng mức cô lập của transaction đang chạy.

8. Đóng phiên và kiểm chứng

Sau khi đã hết hạn:

```
SELECT auction.close_auction(123::bigint,'row_lock');
SELECT auction.close_auction(123::bigint,'row_lock');
SELECT * FROM auction.results WHERE auction_id=123;
SELECT * FROM auction.verify_invariants();
```

Lần đóng đầu chốt leader, lần sau trả lại kết quả cũ. Đóng trước hạn trả `TOO_EARLY`. Không sửa trực tiếp closes_at/status để giả kết quả demo, vì điều đó làm mất ý nghĩa quy tắc. Kịch bản lỗi giữa transaction và cạnh tranh sát hạn đã được tự động hóa trong bộ test, không cần cài trigger gây lỗi vào CSDL demo.

9. Các lỗi thường gặp

- `relation auction... does not exist`: đang chọn sai CSDL hoặc chưa chạy setup.
- Không kết nối PostgreSQL: kiểm tra service, host/port, tên CSDL và `.env`; không dùng mật khẩu GitHub.

- `permission denied`: dùng đúng tài khoản app để gọi hàm, owner cho setup/sinh dữ liệu; không tự cấp quyền ghi rộng để bỏ qua lỗi.

- `BID_TOO_LOW`: tính lại giá tối thiểu từ current_price hoặc start_price.

- `INVALID_INCREMENT`: mức giá không nằm trên lưới của start_price và bid_step.

- `AUCTION_ENDED` dù trạng thái OPEN: tác vụ đóng chưa chạy nhưng thời hạn đã qua; không thể nhận giá nữa.

- Output tiếng Việt lỗi font ở terminal: dùng Windows Terminal/PowerShell hỗ trợ UTF-8; CLI đã cấu hình stdout UTF-8.

Phụ lục C - Kịch bản trình bày

1. Trình bày khoảng 10-15 phút

Phút 0-2: vấn đề và quy tắc

“Đề tài của em là hệ thống đấu giá trực tuyến. Khó khăn là nhiều người trả giá đồng thời và một giao dịch khác đóng phiên. Em cần bảo đảm mỗi lượt được kiểm tra trên trạng thái hợp lệ, giá hiện tại khớp lịch sử và chỉ có một kết quả cuối.”

Nêu giá khởi điểm, bước giá, khoảng nhận `[opens_at,closes_at)`, người được duyệt, người bán không tự trả. Hai yêu cầu cùng giá chỉ một được nhận theo thứ tự xử lý hợp lệ; không hứa theo thứ tự click từ máy khách.

Phút 2-4: mô hình dữ liệu

Mở `docs/EER.md` hoặc sơ đồ SVG kèm dự án. Chỉ ra total/disjoint của tài sản và ba bảng con. Giải thích vì sao seller/bidder là quan hệ theo phiên. Chỉ các khóa quan trọng: registration khóa ghép, request UUID UNIQUE, results khóa chính auction_id.

Nêu `start_price`, `bids.amount`, `current_price`, giá thắng là bốn khái niệm có liên quan nhưng không được dùng lẫn. Cache NULL khi chưa có bid; kết quả winner NULL khi phiên kết thúc không có bid.

Phút 4-6: chuẩn hóa

Mở mục 4 của báo cáo. Minh họa quan hệ rộng lặp lại tên người bán và cấu hình phiên trong mỗi bid. Trình bày `bid_id -> auction_id -> asset_id -> seller_id`; phân rã về các bảng riêng, giữ FK. Hai khóa định danh của quan hệ quan sát bid là bid_id và request_id; quy tắc tăng giá còn tạo khóa nghiệp vụ `(auction_id,amount)`. Phủ tối tiểu tách RHS một thuộc tính, không chứa FD thừa. Lossless vì giao của từng bước tách là khóa quan hệ được tách; dependency preserving vì từng FD nằm trong một quan hệ.

Cache max không tự đúng vì BCNF; đây là dữ liệu lặp xuyên bảng được giữ nhất quán bằng transaction và kiểm tra cuối giao dịch.

Phút 6-9: demo trả giá và đóng phiên

```
.\.venv\Scripts\python.exe -m auction_lab demo --strategy row_lock
```

Đọc các bước theo JSON: nhận lượt đầu, replay cùng UUID, hai người cùng trả một mức chỉ một được nhận, đóng sớm bị từ chối, đóng đúng hạn, đóng lại trả kết quả cũ. Mỗi lần chạy tạo phiên mới nên có thể thực hiện lại.

Mở pgAdmin, chạy `SELECT * FROM auction.verify_invariants();`. Giải thích từng invariant và đối chiếu bid/cache/result của ID vừa demo. Nếu có thời gian làm hai Query Tool theo hướng dẫn để thấy chờ khóa.

Phút 9-12: so sánh và minh chứng

Mở `output/benchmark/summary.csv` hoặc bảng thực đo trong báo cáo. Giữ cùng workload/seed/worker và đổi thứ tự strategy giữa các round. Khóa dòng có hàng đợi trên phiên; SERIALIZABLE có thể abort rồi retry. Đọc cả accepted, replayed, rejected, retries, exhausted/errors và latency. Không kết luận chiến lược nào luôn nhanh hơn từ ba round cục bộ.

Mở `output/test-results.json`: các test quan trọng là cùng giá, UUID đồng thời/liên phiên, bid/close race, chờ khóa qua deadline, lỗi sau INSERT và retry transaction thực. Nêu số test thực trong tệp thay vì học thuộc một con số nếu vừa chạy lại phiên bản mới.

Phút 12-15: đối tượng, OQL và giới hạn

```
.\.venv\Scripts\python.exe -m auction_lab oql
```

Trình bày lớp bắt buộc và truy vấn theo collection. Nói rõ PostgreSQL chạy SQL; OQL là đặc tả học thuật, chương trình chạy bản truy vấn Python tương đương. Kết thúc bằng giới hạn: dữ liệu giả lập, chưa có thanh toán/xác thực, benchmark cục bộ và hotspot một phiên.

2. Câu hỏi thường gặp

Vì sao không chỉ đặt UNIQUE cho người thắng?

UNIQUE ngăn hai dòng nhưng không chọn đúng leader hoặc ngăn một bid được chấp nhận sau khi kết quả đã đọc. Cần giao dịch đóng phối hợp với giao dịch trả giá, cộng PK và kiểm tra winner/max.

Vì sao không chỉ dùng một lệnh UPDATE giá cao nhất?

Nghiệp vụ còn phải kiểm tra đăng ký, người bán, giờ đóng, idempotency; chèn lịch sử và audit phải cùng thành công. Một UPDATE có điều kiện có thể là thành phần thiết kế khác, nhưng không thay thế việc chứng minh toàn bộ transaction.

READ COMMITTED có đủ an toàn không?

Có cho protocol này khi trước mọi quyết định quan trọng đều khóa dòng phiên và mọi đường ghi tuân thủ cùng thứ tự. Chỉ chọn READ COMMITTED mà không khóa/điều phối không tự bảo đảm an toàn.

SERIALIZABLE có loại bỏ mọi khóa không?

Không. Mã phương án B không chủ động FOR UPDATE trên dòng phiên, nhưng PostgreSQL vẫn có khóa ghi, snapshot và SSI. Cạnh tranh có thể dẫn đến serialization_failure; cần retry toàn transaction.

Tại sao retry phải dùng cùng UUID?

Client có thể không biết commit đã thành công nếu kết nối mất ngay sau commit. Dùng lại UUID giúp trả lại bid cũ thay vì tạo thao tác mới. Tạo UUID mới khi retry có thể làm một ý định trở thành nhiều lượt trả giá.

Nếu cùng UUID nhưng người khác hoặc số tiền khác?

Đó là conflict, không được replay thành công tùy tiện. Payload cần so khớp auction_id, bidder_id và amount; UNIQUE toàn cục kết hợp protocol request xử lý cả hai phiên khác nhau.

Request bị từ chối có được cache không?

Không trong thiết kế này. Chỉ lượt chấp nhận có dòng bền vững. Yêu cầu bị từ chối có thể được đánh giá lại khi trạng thái thay đổi, chẳng hạn đăng ký được duyệt. Đây là lựa chọn thiết kế được công bố, không là API ledger lưu mọi kết quả.

Người gửi trước hạn nhưng chờ khóa đến sau hạn thì sao?

Bị từ chối. Deadline dựa trên giờ CSDL sau khi giành quyền xử lý, không dựa trên timestamp máy khách. Dùng clock_timestamp vì now cố định từ đầu transaction.

Chưa có bid thì current_price bằng giá khởi điểm được không?

Có thể thiết kế vậy nhưng dễ lẫn "đã có người trả giá". Đồ án chọn current_price NULL và hiển thị COALESCE(current_price,start_price); giá thắng chỉ lấy từ bid thực.

Vì sao schema có cache mà vẫn nói chuẩn hóa?

Phần logic được phân rã theo FD đạt BCNF. Phần vật lý thêm dữ liệu suy diễn vì hiệu quả đọc. FD nội bảng và dư thừa xuyên bảng là hai điều cần phân biệt; transaction, quyền và invariant giữ cache nhất quán.

Lỗi sau INSERT có làm ID bị bỏ qua không?

Có thể, vì sequence không rollback. ID bị khuyết không chứng minh có bid dở dang; kiểm tra nội dung bid/cache/audit đã commit mới chứng minh tính nguyên tử.

Vì sao dùng trigger hoãn cho lớp con tài sản?

Vì chèn bảng cha rồi mới chèn bảng con trong cùng transaction. Kiểm tra total ngay sau INSERT cha sẽ từ chối dữ liệu cuối giao dịch vốn hợp lệ. Kiểm tra cuối giao dịch vừa cho phép thứ tự chèn vừa bảo đảm đúng một lớp con.

Vì sao benchmark hai strategy có số accepted khác nhau?

Cùng payload không có nghĩa cùng lịch xử lý. Một giá cao vào sớm có thể làm nhiều giá thấp bị từ chối. Vì vậy phải báo cả counts và latency; không ép số accepted giống nhau bằng cách chạy tuần tự giả đồng thời.

OQL chạy trực tiếp trên PostgreSQL được không?

Không. Các câu OQL là đặc tả ODMG để trình bày hướng đối tượng; có bản SQL và mô phỏng Python tương đương cho dữ liệu mẫu. Không đưa OQL vào Query Tool như SQL thực thi.

Superuser có sửa dữ liệu làm sai quy tắc được không?

Tài khoản quản trị tin cậy có thể thay lược đồ hoặc vô hiệu hóa bảo vệ. Đồ án tách owner/app; app chỉ gọi hàm và không ghi bảng trực tiếp. Không tuyên bố chống được người quản trị có toàn quyền.

3. Trước khi nộp

- Bổ sung thông tin sinh viên/lớp/giảng viên theo thực tế vào bản báo cáo nếu mẫu trường yêu cầu.
- Giữ nguyên `.env` ngoài Git; hướng dẫn người chấm cấu hình `.env.example` trên máy khác.
- Chạy `verify`, bộ test và benchmark nếu có thay đổi code; cập nhật bảng số liệu cùng bản PDF.
- Kiểm tra các tệp SQL, báo cáo, sơ đồ, kết quả thực đo và hướng dẫn được đưa vào bản nộp.
- Chuẩn bị giải thích các quy tắc bằng lời của mình và chạy lại demo, không chỉ đọc báo cáo.